

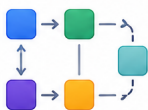
Debugging AI Pipelines



Your RAG app gave a wrong answer. No error, no stack trace. Where do you even start?



The Problem



Failure Modes



Traditional
Debugging Trap



Structured
Logging



Correlation
IDs



What to Log



Log Levels



Storage &
Retention



Observability



Debugging
Workflow



Tools & Stack



Key Takeaways



Find issues
faster



Understand
what's going wrong



Make debugging
repeatable & reliable



Ship better
AI products



Reduce time
to resolution



AI Coach John

Follow



PROITBRIDGE
Solve For Better Future

Repost

! THE PROBLEM

Traditional debugging assumes failures announce themselves

TRADITIONAL BUG

```
TypeError: 42
↑ location
↑ reproducible
↑ deterministic
```

✓ set a breakpoint

AI PIPELINE BUG

```
(returns a
fluent, wrong
answer)
↑ no location
↑ non-determ.
```

✗ breakpoints don't help

🚫 **Failures are silent** — returns `200 OK` with a wrong answer

🔗 **Many stages, one output** — retrieval? prompt? model?

🎲 **Non-reproducible** — rerun it and you may get a different answer

👁️ **Production is invisible** — can't debug a request from 3 hours ago



AI Coach John

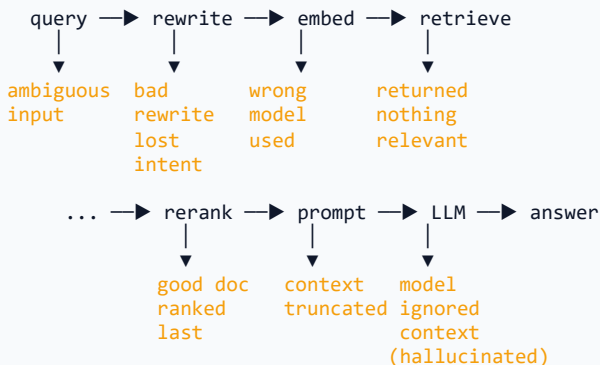
Follow



PROITBRIDGE
Solve For Better Future

↺ ↻ Repost

Six stages, **six different failure modes**



ONE wrong answer. SEVEN possible causes.
Without logs, you're guessing which.

Not "why is the answer wrong?" but "at which stage did it stop being right?" Every logging decision exists to answer that question.



Why `print()` fails you when you need it

WORKS

- ✓ local, one request
- ✓ while you watch it run

FAILS

- ✗ in prod (no stdout you can read)
- ✗ concurrent requests
- ✗ can't filter by severity
- ✗ can't correlate a chain

The concurrency problem:

```
stdout (3 simultaneous requests)

got docs: [doc_a, doc_b]
got docs: [doc_x]
score: 0.91
got docs: [doc_m, doc_n]
score: 0.42
score: 0.88
  ▲ which score belongs to which request?
  unknowable.
```



Structured Logging: **objects**, not sentences

× UNSTRUCTURED

```
"Retrieved 5 docs for query 'refund policy' in 340ms"
```

↳ to find slow retrievals you'd have to regex-parse this

✓ STRUCTURED

```
{
  "event": "retrieval_complete",
  "query": "refund policy",
  "doc_count": 5,
  "latency_ms": 340,
  "top_score": 0.91,
  "request_id": "req_a3f9"
}
```

↳ query it: WHERE latency_ms > 500 AND top_score < 0.7

```
import structlog
log = structlog.get_logger()

log.info("retrieval_complete",
        query=query, doc_count=len(docs),
        top_score=scores[0], latency_ms=elapsed,
        request_id=request_id)
```



Correlation IDs: **Stitching a request together**

One ID that follows a request through every stage.

WITHOUT ID

retrieval_complete
rerank_complete
retrieval_complete
llm_call_complete
rerank_complete
scrambled across users

WITH ID

req_a3f9 • query_rcvd
req_a3f9 • retrieve_done
req_a3f9 • rerank_done
req_a3f9 • llm_complete
req_a3f9 • answer_sent
one filter = whole story

```
from contextvars import ContextVar
request_id_var = ContextVar("request_id")

def handle_request(query):
    request_id_var.set(f"req_{uuid.uuid4().hex[:8]}")

    # bind once - applies to EVERY log line in this request
    structlog.contextvars.bind_contextvars(
        request_id=request_id_var.get()
    )

    docs = retrieve(query) # logs inside carry request_id
```



What to log at every stage

```
# 1 Request & 2 Rewrite
log.info("query_received", query=q, session_id=sid)
log.info("query_rewritten", original=q, rewritten=new_q)

# 3 Retrieval & 4 Rerank
log.info("retrieval_complete", doc_ids=[d.id for d in docs],
        scores=scores, latency_ms=elapsed)
log.info("rerank_complete", dropped=len(docs)-len(reranked))

# 5 Prompt & 6 LLM
log.info("prompt_built", truncated=was_truncated)
log.info("llm_complete", input_tok=usage.in, out_tok=usage.out,
        cost_usd=cost, finish_reason=resp.finish_reason)
```

THE FIELDS THAT PAY FOR THEMSELVES

doc_ids + scores	→ "was the right doc retrieved?"
before/after ids	→ "did reranking help or hurt?"
truncated flag	→ "did context silently get cut off?"
finish_reason	→ "did it stop early on max_tokens?"
token counts	→ cost attribution + budget



Log levels that **actually** mean something

DEBUG

full prompts, full text, raw model output
— off in prod, invaluable locally

INFO

stage completions, counts, scores, tokens
— the default production level

WARNING

degraded but working (retrieval low, fallback)

ERROR

request failed (parse error, outage)

CRIT

systemic (db dead, model dead)

⚠ THE MOST UNDERUSED LEVEL IS WARNING

These are silent failures that return 200 OK to the user:

retrieval top_score = 0.31 → "weak_retrieval"

context truncated by 4k → "context_truncated"

fell back to secondary → "model_fallback_used"

judge flagged low score → "low_faithfulness"

Without a WARNING log, nobody ever finds out.



Logs are a data-leak surface

✗ NEVER LOG

API keys
full user PII
raw documents
full prompts (in prod)
credit cards, health

✓ LOG INSTEAD

key_present: true
user_id: hash_a3f
doc_ids + scores
prompt_tokens: 4k
redacted: "[CARD]"

```
def redact(text: str) -> str:
    for label, pattern in PATTERNS.items():
        text = re.sub(pattern, f"[{label.upper()})", text)
    return text

log.info("query_received", query=redact(query))
```



Log enough to reconstruct what happened, never enough to reconstruct who it happened to.



AI Coach John

Follow



PROITBRIDGE
Solve For Better Future

Repost

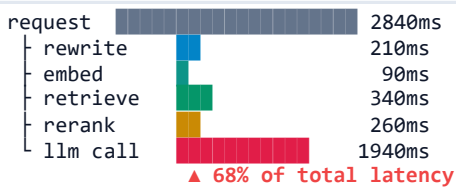
Tracing: The real upgrade (The whole story)

Logs are events. Traces are the whole story with timing.

LOGS (FLAT LIST OF MOMENTS)

```
retrieval done
rerank done
llm done
answer sent
```

TRACE (NESTED, TIMED, CAUSALLY LINKED)



- 🕒 **Where the time went** — instantly obvious bottlenecks
- 👤 **Parent-child causality** — failing sub-steps attach to parents



Tracing in Practice

```
# — LangSmith (near zero setup for LangChain) —  
os.environ["LANGCHAIN_TRACING_V2"] = "true"  
# every step is now traced automatically
```

```
# — Langfuse (open-source, self-hostable) —  
from langfuse.decorators import observe  
  
@observe()  
def retrieve(query: str):  
    return vector_store.search(query, k=5)
```

```
# — OpenTelemetry (vendor-neutral, fits existing infra) —  
with tracer.start_as_current_span("retrieval") as span:  
    docs = retrieve(query)  
    span.set_attribute("doc_count", len(docs))
```

using LangChain/Graph?

—► **LangSmith**

want open-source/self-host?

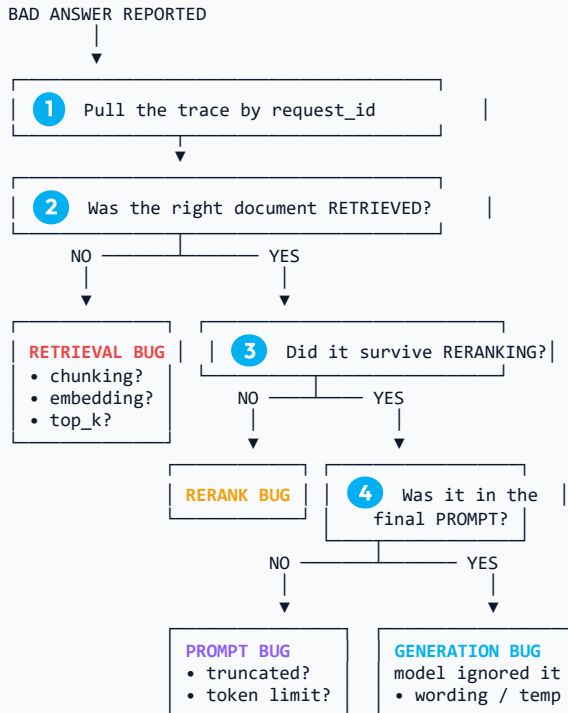
—► **Langfuse**

already have Datadog/Grafana?

—► **OpenTelemetry**



A systematic method instead of guessing



Debugging Retrieval (Most hallucinations)

```
def debug_retrieval(query, expected_doc_id):  
    # 1. Is it in the index?  
    if not vector_store.get(expected_doc_id):  
        print("❌ Not indexed – check ingestion")  
  
    # 3. What beat it, and by how much?  
    results = vector_store.similarity_search_with_score(query)  
    for rank, (doc, score) in enumerate(results, 1):  
        marker = "★" if doc.id == expected_doc_id else " "  
        print(f"{marker} {rank}. {score} {doc.id}")
```

RETRIEVAL DECISION TABLE

not in index at all	ingestion pipeline failed
scores low (< 0.5)	chunking split answer, or bad query
scores well but ranks 8th	top_k too small
scores clustered ~0.7	embed model bad → try different one
exact term missing	need hybrid BM25 + dense



Debugging the Prompt (Log what arrived)

The prompt you *think* you sent and the one that *arrived* are often different due to truncation.

```
# The silent killer: did any chunk get cut off?
for i, doc in enumerate(docs):
    if doc.text not in prompt:
        print(f"⚠️ Chunk {i} was TRUNCATED out of prompt")
```

TOKEN ACCOUNTING

WHAT YOU WROTE

sys prompt	500 tok
10 context chunks	6000 tok
user question	40 tok
total	6540 tok

WHAT THE MODEL GOT

sys prompt	500 tok	✓
6 context chunks	3600 tok	⚠️ 4 drop
user question	40 tok	✓
total	4140 tok	
▲ silently truncated to fit the answer was in chunk 8		

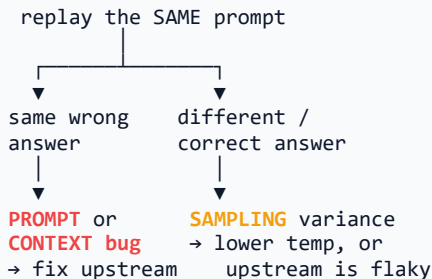


Replay & Reproduction (The one-off failure)

Non-determinism makes "it happened once" the hardest bug. Capture state to replay it.

```
def capture_failure_case(request_id, state):  
    # Save EVERYTHING needed to reproduce  
    json.dump({  
        "query": state["query"],  
        "retrieved_text": [d.text for d in state["docs"]], # text, not IDs  
        "prompt": state["prompt"],  
        "temperature": state["temperature"],  
        "seed": state.get("seed"),  
    }, open(f"debug/{request_id}.json", "w"))
```

REPLAY TELLS YOU WHICH HALF TO BLAME



Production Monitoring: Detect before complaints

```
# Emit metrics alongside logs, on every request
metrics.histogram("rag.retrieval.top_score", scores[0])
metrics.histogram("rag.latency_ms", total_latency)
metrics.counter("rag.context_truncated", 1 if trunc else 0)
metrics.counter("rag.no_results", 1 if not docs else 0)
```

ALERTS WORTH SETTING

mean top_score < 0.65 (1h)	retrieval quality drop
no_results rate > 5%	index or query problem
p95 latency > 3s	perf regression
truncation rate > 10%	chunks too big
cost/hour > \$X	runaway loop or abuse

✓ Each of these fires BEFORE users start complaining.



Sampling: Full logging gets expensive fast

× LOG EVERYTHING

every request
full prompts
full chunk text
TBs/month 📦

▼ SAMPLED

100% of errors / downvotes
100% of slow requests
10% normal requests
0% full chunk text

```
def should_log_verbose(request) -> bool:
    if request.had_error: return True # always
    if request.feedback == "negative": return True # always
    if request.latency_ms > 3000: return True # always
    return random.random() < 0.10 # 10% baseline
```

You don't need verbose logs for requests that went fine — you need them for ones that didn't, and you can identify most automatically.





Complete Observability Setup Assembled

```
@contextmanager
def traced_stage(name, **fields):
    start = time.perf_counter()
    try:
        yield
    except Exception as e:
        log.error(f"{name}_failed", error=str(e), lat=..., **fields)
        raise
    else:
        log.info(f"{name}_complete", lat=..., **fields)

def run_pipeline(query):
    bind_contextvars(request_id=f"req_{uuid.uuid4().hex[:8]}")
    with traced_stage("retrieval"): docs = retrieve(query)
    with traced_stage("rerank"):    ranked = rerank(query, docs)
    with traced_stage("gen"):      return generate(query, ranked)
```

THE FIVE LAYERS

- 1 **structured logs** —> queryable events
- 2 **correlation IDs** —> stitch a request together
- 3 **tracing** —> timing + nesting + causality
- 4 **metrics/alerts** —> detect before users complain
- 5 **failure capture** —> replay a one-off bug





What goes wrong with AI observability

- ⚠ Using `print()` past the prototype stage
- ⚠ Logging full prompts and PII into production logs
- ⚠ No correlation ID — logs unusable under concurrency
- ⚠ Only logging errors (silent wrong answers stay invisible)
- ⚠ No WARNING level for degraded-but-working states
- ⚠ Logging everything at full verbosity until the bill arrives

SYMPTOM

"can't reproduce it"

"logs are useless"

"we found out from a user"

"the log bill is huge"

"answer was wrong, no clue"

USUALLY MEANS

no failure capture / replay

unstructured strings

no metrics or alerts

no sampling strategy

scores/prompt not logged

